DEMONSTRAÇÃO DE COMPETÊNCIA DE PROJETO DE ANÁLISE E FLUXO DE DADOS OLAP E ETL

DEMONSTRATION OF COMPETENCE IN OLAP AND ETL DATA ANALYSIS AND DATA FLOW DESIGN

Kayo Gabriel Soares da Silva[1]

RESUMO: O trabalho apresentado aborda técnicas de modelagem de banco de dados com **SQL Server** e **PostgreSQL** para consultas analíticas e automação de processos com a ferramenta **Apache Airflow** em um ambiente **Docker**.

Palavras-chave: SGBD; OLTP; OLAP; Fluxo de tarefas; Automação.

ABSTRACT: The work presented covers database modeling techniques using **SQL Server** and **PostgreSQL** for analytical queries, as well as process automation with the **Apache Airflow** tool within a **Docker** environment.

Keywords: DBMS; OLTP; OLAP; Task flow; Automation.

1 INTRODUÇÃO

Com um ambiente robusto de se trabalhar com modelagem de dados é normal que com o tempo os processos manuais vão sendo automatizados. O **Python** vem se destacando no mercado de dados por ele ser um canivete suíço e com uma forte adesão dos analistas de dados, que conta com tecnologias maduras sendo uma delas o **Pandas** e para automação de processos o **Apache Airflow** que permite o usuário agendar acionamentos de tarefas para que de tempo em tempo elas possam extrair dados de um banco transacional (OLTP), organizar, tratar e carregar os resultados em um banco de dados analítico (OLAP). Esse foi um exemplo e o caso de uso pode variar bastante dependendo do objetivo desejado. Trabalho disponibilizado para revisão no seguinte endereço: https://github.com/Kayo-Gabriel/ETL_OLAP_COMPETENCIA.git

2 REVISÃO DE LITERATURA

A modelagem de dados eficiente é a base para a tomada de decisão estratégica. Para isso, utiliza-se o conceito de processamento analítico (OLAP), que consolida informações e otimiza consultas para *dashboards*. Nesse cenário, sistemas de gerenciamento relacionais, como o **SQL Server** e o **PostgreSQL**, atuam como os pilares de infraestrutura, servindo tanto para o armazenamento de origem quanto para a estruturação do banco de dados analítico.

Para que os dados transitem entre esses bancos, aplica-se o processo de Extração, Transformação e Carga (ETL). O **Apache Airflow** destaca-se como a principal plataforma para automatizar esse fluxo, utilizando **Python** para organizar *pipelines* em Grafos Acíclicos Direcionados (DAGs). Contudo, como o **Airflow** não isola nativamente suas tarefas, aliar essa orquestração a ambientes em contêineres,

[1] Unisales. Vitória/ES, Brasil. E-mail: KAYO.SILVA@souunisales.com.br

utilizando o **Docker**, torna-se a solução ideal para garantir que toda a operação funcione com estabilidade e sem sobrecarga do sistema.

3 METODOLOGIA

O trabalho apresentado foi desenvolvido utilizando **Docker** para preparar e isolar em um container as dependências das ferramentas e o ambiente de trabalho para não misturar com as dependências do sistema operacional da máquina. Para o banco de dados transacional (OLTP) foi utilizado o **SQL Server** da Microsoft onde foi feito um backup de um AdventureWorks oficial para estar populando o banco com o objetivo de estar utilizando esses dados em operações futuras. Para a modelagem e desenvolvimento do banco de dados analítico (OLAP) foi utilizado o SGBD **PostgreSQL** na modelagem Star Schema do Data Warehouse. Na preparação dos processos de ETL foi utilizado a linguagem **Python** juntamente com algumas de suas dependências essenciais para esse trabalho, sendo elas o **Pandas** que permite criar Data Frames em memória para estar trabalhando os dados e suas granularidades e o **Airflow** que automatiza um conjunto inteiro de tarefas através de agendamentos e disponibiliza um painel visual para acompanhar os resultados de processamento das DAGs. Com os scripts de ETL preparados e ligados ao **Airflow,** o Data Warehouse foi populado seguindo uma estratégia de incremento através de uma variável do **Airflow** que permite consultar quando foi a última operação e só processa os registros cujo a data e o tempo seja maior que o registrado no **Airflow** para ai o pipeline ativar e carregar na tabela fato, assim evitando processamento desnecessário e economizando recursos computacionais em tabelas com grandes concentrações de dados. Nas tabelas dimensões foi utilizado a mesma estratégia que na tabela fato para manter a dimensão funcionários como incremental pois ela concentra mais dados. As outras dimensões não foram feitas seguindo a operação incremental por conta da quantidade baixa de dados que é concentrado nelas, onde o banco gasta mais recursos pra fazer a operação de incremento do que uma inserção de sobre escrita na dimensão.

4 RESULTADOS E DISCUSSÃO

O resultado alcançado foi uma “esteira” rolante de tarefas automatizadas, execuções práticas e bem sucedidas das DAGS, modelagens, praticas, população do Data Warehouse para a execução das KPIs de negócios referente ao setor de Recursos Humanos (RH).

Dicionário de dados

DDATE

Coluna	Tipo de Dado	Restrição	Descrição
DateID	serial	Primary Key	Identificador único da data
FullDate	date	Not null	Data completa
Day	smallint	Not null	Dia do mês
Month	smallint	Not null	Mês do ano
Year	smallint	Not null	Ano
Semester	smallint	Not null	Semestre do ano

Fonte: Produção própria

DTIME

Coluna	Tipo de Dado	Restrição	Descrição
TimeID	serial	Primary Key	Identificador único do horário
FullTime	time	Not null	Horário completo
Hour	smallint	Not null	Hora
Minute	smallint	Not null	Minuto
Second	smallint	Not null	Segundo

Fonte: Produção própria

DDEPARTMENT

Coluna	Tipo de Dado	Restrição	Descrição
DepartmentID	int	Primary Key	Identificador único do departamento
Name	varchar(100)	Not null	Nome do departamento
GroupName	varchar(100)	Not null	Nome do grupo organizacional

Fonte: Produção própria

DEMPLOYEE

Coluna	Tipo de Dado	Restrição	Descrição
EmployeeID	int	Primary Key	Identificador único do funcionário
NationalIDNumber	varchar(15)	Not null	Número de identificação nacional (ex: CPF/RG)
LoginID	varchar(256)	Not null	Credencial de login do funcionário
OrganizationNode	varchar(892)	Nenhuma	Nó de organização hierárquica
OrganizationLevel	smallint	Nenhuma	Nível hierárquico na organização
JobTitle	varchar(50)	Not null	Cargo do funcionário
MaritalStatus	char(1)	Not null	Estado civil
Gender	char(1)	Not null	Gênero
BirthDate	date	Not null	Data de nascimento
HireDate	date	Not null	Data de contratação
SalariedFlag	smallint	Check (0 ou 1)	Indicador se é assalariado
CurrentFlag	smallint	Check (0 ou 1)	Indicador se o funcionário está ativo atualmente

Fonte: Produção própria

DJOB_CANDIDATE

Coluna	Tipo de Dado	Restrição	Descrição
JobCandidateID	int	Primary Key	Identificador único do candidato
Resume	text	Nenhuma	Currículo do candidato

Fonte: Produção própria

DPAY_HISTORY

Coluna	Tipo de Dado	Restrição	Descrição
PayHistoryID	int	Primary Key	Identificador único do histórico de pagamento
PayFrequence	smallint	Not null	Frequência do pagamento

Fonte: Produção própria

DSHIFT

Coluna	Tipo de Dado	Restrição	Descrição
ShiftID	int	Primary Key	Identificador único do turno
Name	varchar(100)	Not null	Nome do turno
StartTime	time	Not null	Horário de início do turno
EndTime	time	Nenhuma	Horário de término do turno

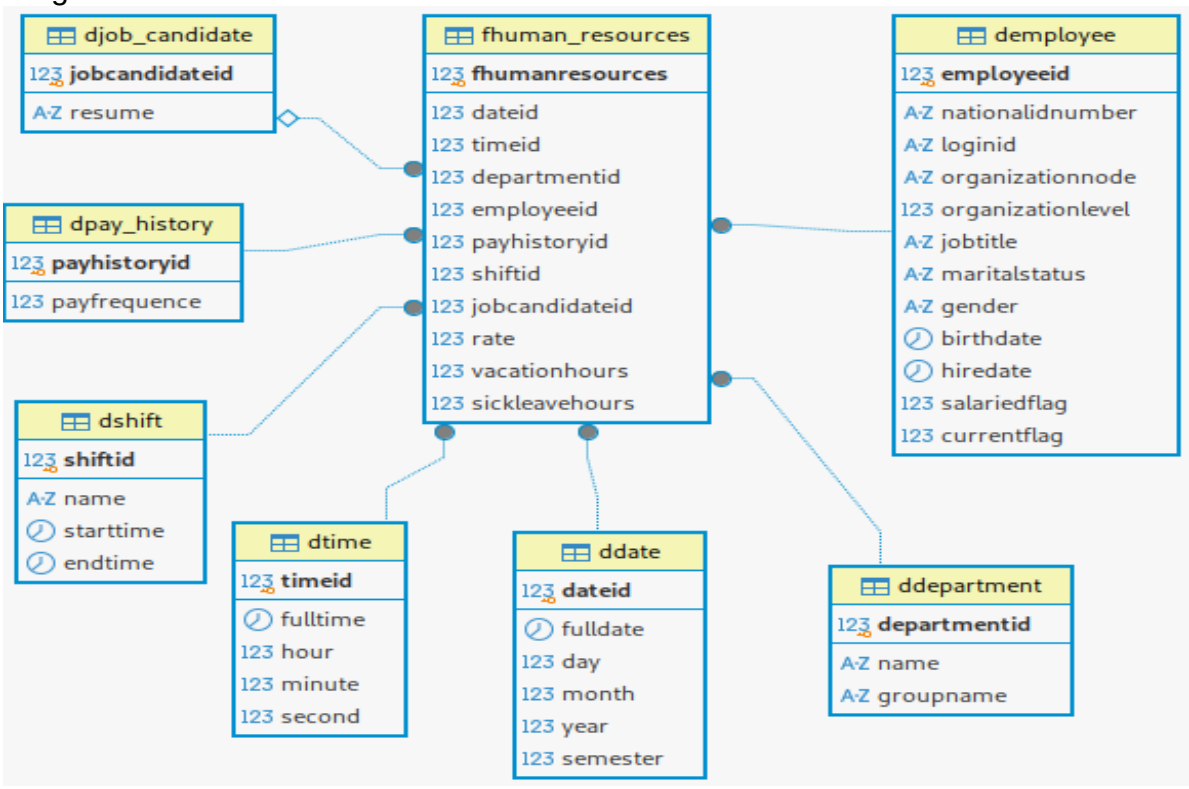
Fonte: Produção própria

FHUMAN_RESOURCES

Coluna	Tipo de Dado	Restrição	Descrição
FHumanResources	serial	Primary Key	Identificador único do registro fato
DateID	int	FK -> ddate(DateID), Not null	Chave estrangeira para a data
TimeID	int	FK -> dtime(TimeID), Not null	Chave estrangeira para o horário
DepartmentID	int	FK -> ddepartment(DepartmentID), Not null	Chave estrangeira para o departamento
EmployeeID	int	FK -> demployee(EmployeeID), Not null	Chave estrangeira para o funcionário
PayHistoryID	int	FK -> dpay_history(PayHistoryID), Not null	Chave estrangeira para o histórico de pagamento
ShiftID	int	FK -> dshift(ShiftID), Not null	Chave estrangeira para o turno de trabalho
JobCandidateID	int	FK -> djob_candidate(JobCandidateID)	Chave estrangeira para o candidato à vaga
Rate	numeric(15,2)	Not null	Taxa/Valor do salário por hora
VacationHours	smallint	Not null	Total de horas de férias
SickLeaveHours	smallint	Not null	Total de horas de licença médica

Fonte: Produção própria

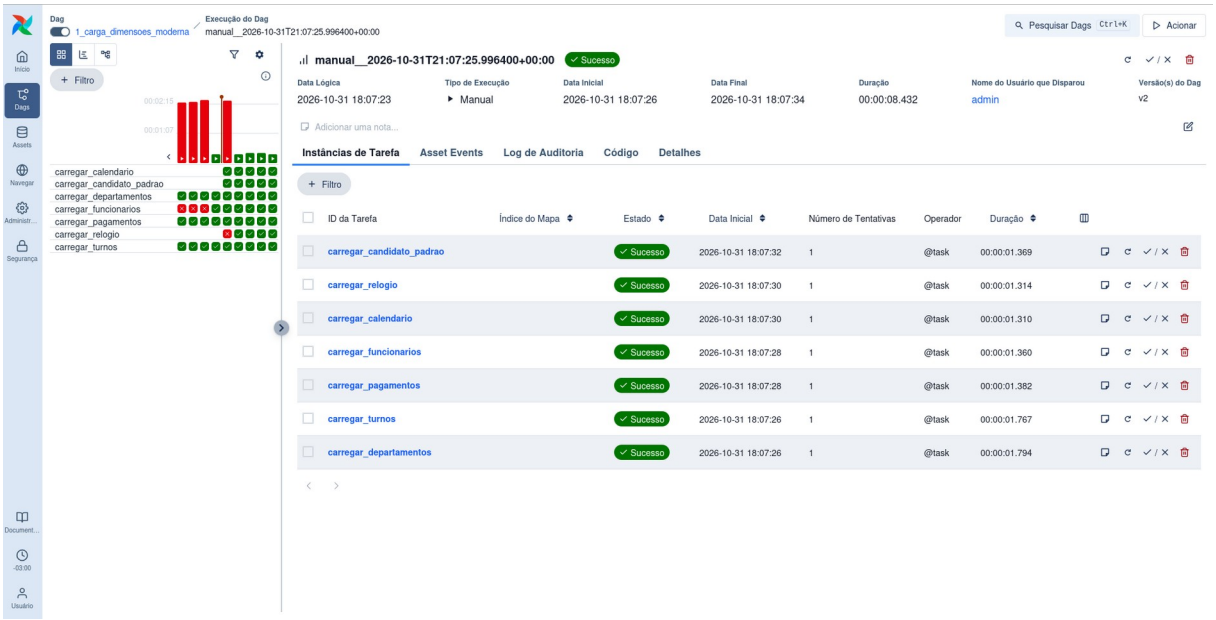
Diagrama Relacional



Fonte: Produção própria

Orquestração de processos - Tabelas Dimensões

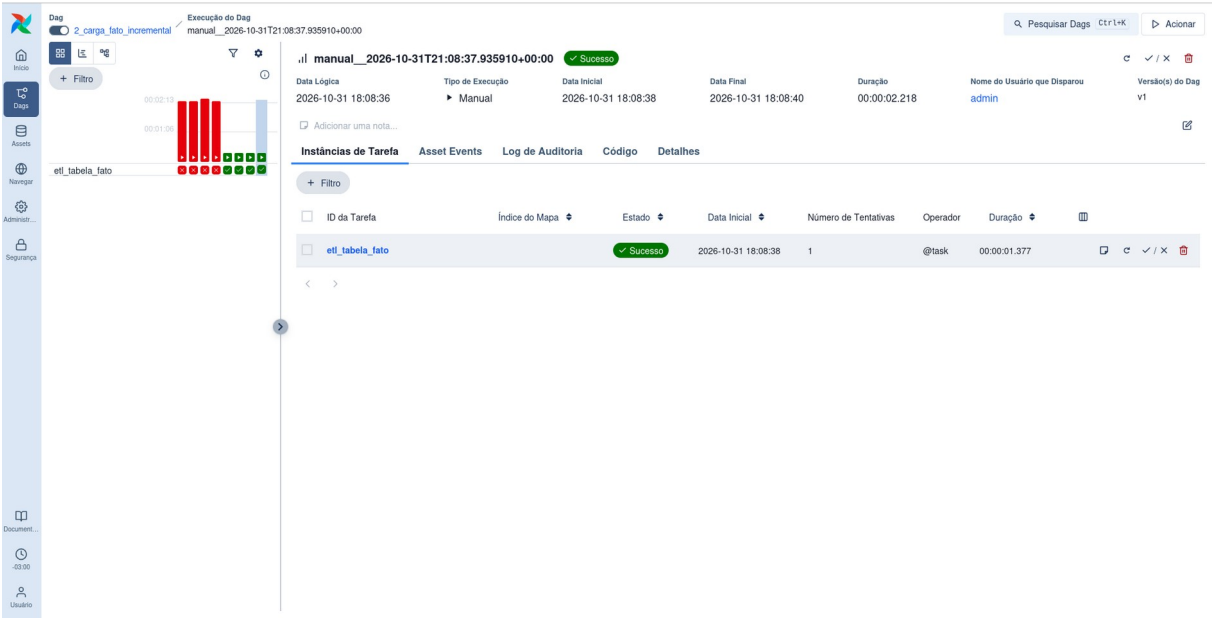
A imagem a seguir mostra o histórico de acionamentos das tarefas de ETL para popular as tabelas dimensões, comprovando o funcionamento da pipeline de execuções.



Fonte: Produção própria

Orquestração de processos - Tabela Fato

A imagem a seguir mostra o histórico de acionamento da pipeline de população da tabela fato do Data Warehouse, comprovando assim o funcionamento da pipeline e efetivando a extração dos dados no banco de dados transacional (OLTP) SQL Server e carregando a tabela fato do banco de dados analítico (OLAP) feito em PostgreSQL.



Fonte: Produção própria

Granularidade

A tabela fato de recursos humanos foi feita para armazenar histórico de alteração de salários contratuais dos empregados, tendo como granularidade a representação de uma alteração de taxa salarial de um funcionário especifico lincado ao seu departamento e turno de trabalho.

No processo de modelagem foi aceita a desnormalização dos dados pra obter ganho de desempenho analítico nas consultas de KPIs. A modelagem Star Schema é muito utilizada no mercado analítico porque quanto menos relacionamentos entre tabelas, mais desempenho de consultas é ganho comparado a modelagem tradicional ou Snow Flake.

Estratégia da ETL incremental orquestrada

A orquestração feito na ferramenta **Airflow** foi separada em duas etapas sendo a primeira etapa responsável por popular as tabelas dimensões onde foi aplicado a estratégia incremental somente na dimensão com maior volume de dados para evitar desperdício computacional, onde foi utilizado a variavel do **Airflow** para identificar a última data e hora registrada e a partir dai o processamento dos novos dados que serão populados na tabela. Já nas dimensões menos volumosas não foi aplicado o processo incremental porque o banco gasta mais recursos fazendo o processo incremental do que só processar os dados. Na tabela fato foi aplicado o processo incremental, pois nela se concentra maior parte dos dados do Data Warehouse e a estratégia adotada foi a mesma da tabela dimensão de funcionários (DEMPLOYEE).

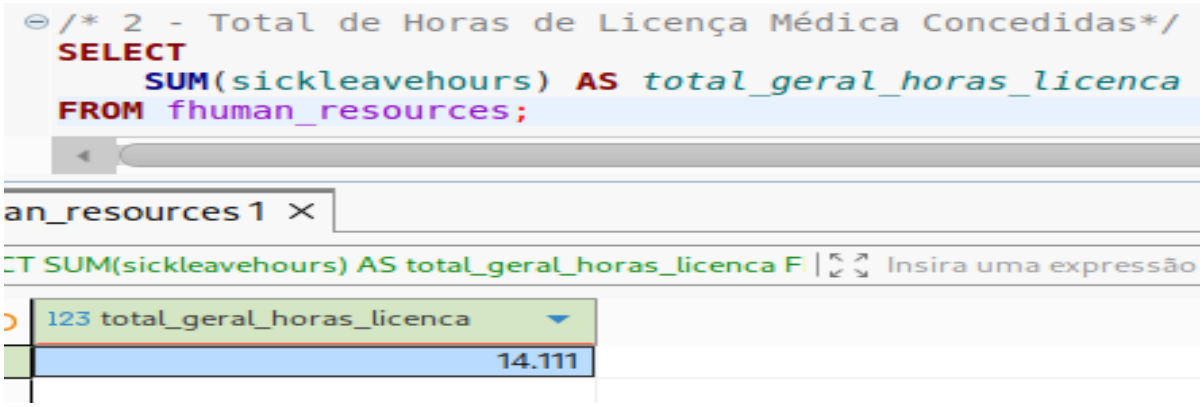
KPIs

1 – Quantidade total de registros de RH na fato



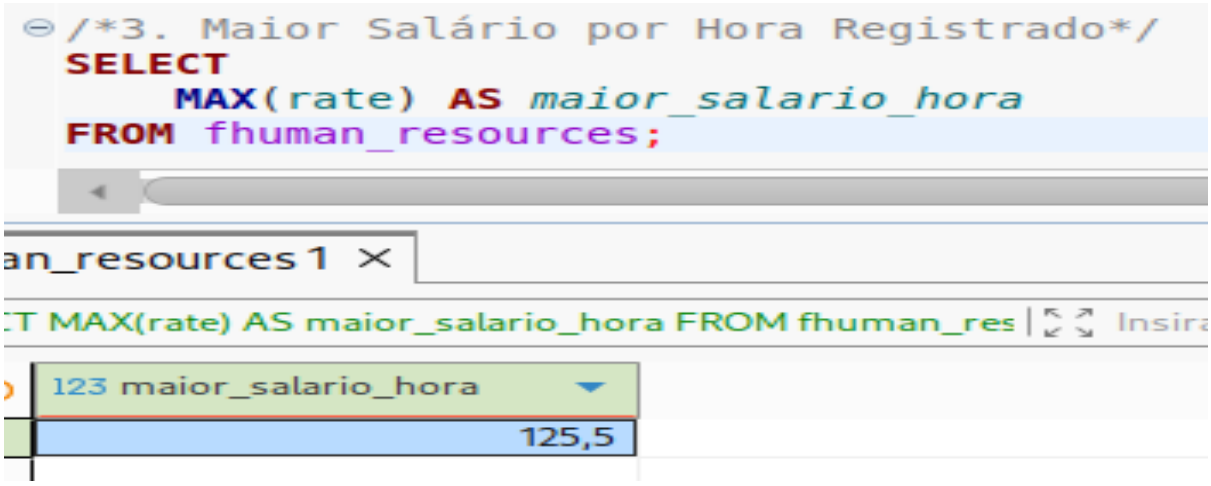
Fonte: Produção própria

2 – Total de horas de licença médica concedidas



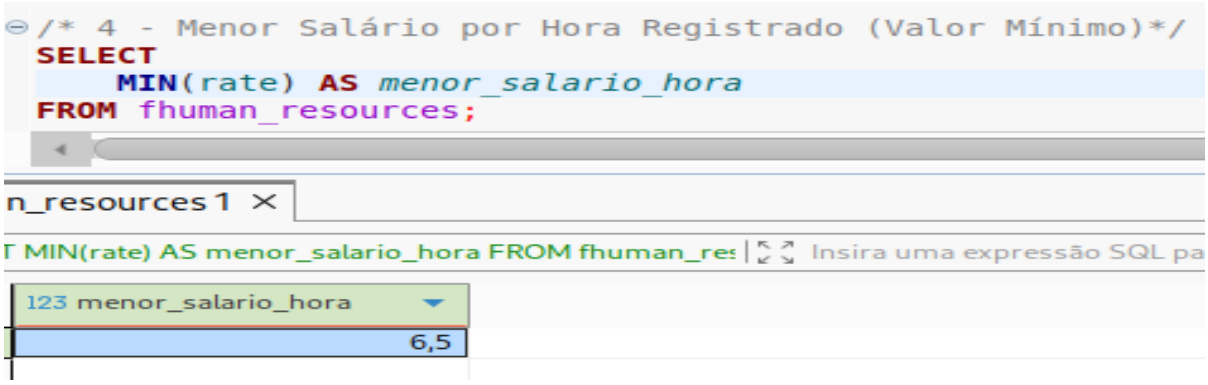
Fonte: Produção própria

3 – Maior salário por hora registrado



Fonte: Produção própria

4 – Menor salário por hora registrado



Fonte: Produção própria

5 – Média geral de horas de férias acumuladas

```sql
/* 5 - Média Geral de Horas de Férias Acumuladas*/
SELECT
    AVG(vacationhours) AS media_geral_horas_ferias
FROM fhuman_resources;
```

an_resources 1

T AVG(vacationhours) AS media_geral_horas_ferias FF | Insira uma express

123 media_geral_horas_ferias
48,8924050633

Fonte: Produção própria

6 – Soma de horas de licença médica por departamento

```sql
/* 6 - Soma de Horas de Licença Médica por Departamento*/
SELECT
    d.name AS departamento,
    SUM(f.sickleavehours) AS total_horas_licenca
FROM fhuman_resources f
JOIN ddepartment d ON f.departmentid = d.departmentid
GROUP BY d.name;
```

artment 1

T d.name AS departamento, SUM(f.sickleavehours) AS | Insira uma expressão SQL pa

A-Z departamento	123 total_horas_licenca
Quality Assurance	366
Sales	605
Production Control	339
Research and Development	216
Document Control	291
Engineering	129
Executive	129
Purchasing	652
Finance	495
Facilities and Maintenance	450
Information Services	545
Human Resources	273
Tool Design	311
Shipping and Receiving	405
Production	8.449
Marketing	456

Fonte: Produção própria

7 – Média salarial por gênero

```sql
/* 7 - Média Salarial por Gênero*/
SELECT
    e.gender AS genero,
    AVG(f.rate) AS media_salarial_hora
FROM fhuman_resources f
JOIN demployee e ON f.employeeid = e.employeeid
GROUP BY e.gender;
```

loyee 1 ×

T e.gender AS genero, AVG(f.rate) AS media_salarial_h | Insira uma expr

A-Z genero	123 media_salarial_hora
M	17,0026315789
F	19,7184090909

Fonte: Produção própria

8 – Total de funcionários por estado civil

```sql
/* 8 - Total de Funcionários por Estado Civil*/
SELECT
    maritalstatus AS estado_civil,
    COUNT(*) AS total_colaboradores
FROM demployee
GROUP BY maritalstatus;
```

loyee 1 ×

T maritalstatus AS estado_civil, COUNT(*) AS total_col | Insira uma ex

A-Z estado_civil	123 total_colaboradores
M	146
S	144

Fonte: Produção própria

9 – Total de departamentos por grupo organizacional

```sql
/* 9 - Total de Departamentos por Grupo Organizacional*/
SELECT
    groupname AS grupo_departamento,
    COUNT(*) AS total_departamentos
FROM ddepartment
GROUP BY groupname;
```

artment 1 ×

T groupname AS grupo_departamento, COUNT(*) AS · | Insira uma expressão SQL pa

A-Z grupo_departamento	123 total_departamentos
Quality Assurance	2
Inventory Management	2
Executive General and Administration	5
Research and Development	3
Sales and Marketing	2
Manufacturing	2

Fonte: Produção própria

10 – Média de horas de férias por turno de trabalho

```
/* 10 - Média de Horas de Férias por Turno de Trabalho*/
SELECT
    s.name AS turno,
    AVG(f.vacationhours) AS media_horas_ferias
FROM fhuman_resources f
JOIN dshift s ON f.shiftid = s.shiftid
GROUP BY s.name;
```

```
s.name AS turno, AVG(f.vacationhours) AS media_h
```

Insira uma expressão SQL

A-Z turno	123 media_horas_ferias
Day	46,7376237624
Night	56,5769230769
Evening	49,4677419355

Fonte: Produção própria

5 CONSIDERAÇÕES FINAIS

No mundo dos dados é importante ter conhecimento de práticas de modelagem e simplificação de processos para conseguir trabalhar melhor com as estruturas de dados disponíveis. Um banco transacional (OLTP) possui suas otimizações e técnicas de se trabalhar, com um banco de dados analítico não é diferente. Nesse trabalho o **PostgreSQL** permitiu que a modelagem Star Schema fosse aplicado na prática como também permitiu o desenvolvimento das KPIs que respondem perguntas relacionadas ao RH que foi o tema escolhido. O ambiente **Docker** somados com as ferramentas que o **Python** deixa a disposição fez com que o ambiente ficasse robusto permitindo através do **Airflow** e **Pandas** trabalhar com os dados de forma mais simples e com automatizações que permite definir uma frequência de acionamento, assim, de tempo em tempos os processos de uma ETL são realizadas de forma automática, sendo possível também acompanhar e interagir com a ferramenta através de uma interface visual web do **Airflow** que roda localmente no navegador web.

REFERÊNCIAS

ALVES, James. **unisales.etl.20262.** GitHub, 2026. Disponível em: https://github.com/Jamesasj/unisales.etl.20262.git. Acesso em: 31 out. 2026.

ALVES, James. **Utilizar práticas para realização de consultas avançadas**. Vitória: UniSales, 2026. Material didático disponibilizado no Ambiente Virtual de Aprendizagem (AVA). Acesso em: 3 out. 2026.

BARBOZA, Lucas Daniel. **Arquitetura de banco de dados**. Vitória: UniSales, 2026. Material didático disponibilizado no Ambiente Virtual de Aprendizagem (AVA). Acesso em: 3 out. 2026.

BARBOZA, Lucas Daniel. **Sistemas de Gerenciamento de Banco de Dados**. Vitória: UniSales, 2026. Material didático disponibilizado no Ambiente Virtual de Aprendizagem (AVA). Acesso em: 3 out. 2026.